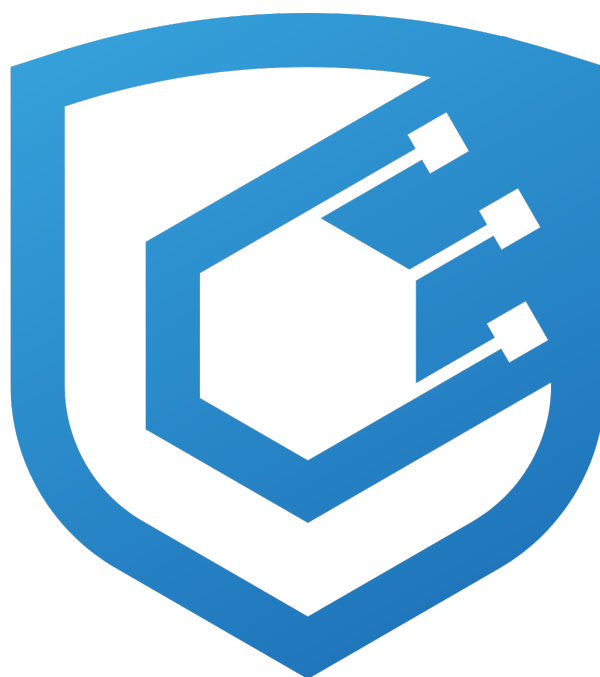




Securitize SVM Jump Router Market Account Audit Report

Prepared by [Cyfrin](#)
Version 2.0

Lead Auditors

[JesJupyter](#)
[Nate](#)

September 29, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Review Assumptions	3
6.2	Centralization Risks	3
6.3	Defensive Improvements	3
7	Findings	5
7.1	Informational	5
7.1.1	The added market account pushes default swap transactions past the 1232-byte limit	5
7.1.2	Pre-existing off-chain SDK transaction-construction inconsistencies	9

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Securitize SVM Jump Router is an on-chain swap-routing program previously [audited](#) by Cyfrin for the Solana Virtual Machine. It intermediates trades between regulated digital security tokens (the “asset” mint) and a paired liquidity token (the “liquidity” mint, typically a stablecoin), executing the underlying trade against a Jump Prop AMM pool.

This engagement reviewed the BC-2563 changes, which add swaps against market-backed Prop AMM V3 pools. The router reads the market binding from the pool account. When the pool is market-backed, it checks the bound market-oracle account and passes that account into both the quote and the swap. Pools that are not market-backed stay on the previous path. The review covered that check, the existing swap paths, and the extra account’s effect on off-chain transaction size.

5 Audit Scope

The audit focus was [changes](#) for BC-2563 spanning commits:

- 98c4747648c9a93231859495d0fe287acb882d4f
- 9279658bdd7db20ccbff707f132a9a317f426ba5
- b8cca7e34747e794f432cedaeabc754a62eab07f
- fe3149106b9b09b774b6a56cc445f6d1717443d0
- 03867a9be1f52000bc8ab0a8d2b4cd6d01adf9f3
- e87d62e327c92b86b617eaa57f274f827429783c (merge PR2 -> dev)

The files in scope were limited to:

```
programs/bc-solana-jump-router-sc/src/instructions/cpi/prop_amm/request_for_quote.rs
programs/bc-solana-jump-router-sc/src/instructions/cpi/prop_amm/swap.rs
programs/bc-solana-jump-router-sc/src/instructions/operator/execute_swap.rs
programs/bc-solana-jump-router-sc/src/instructions/swap_accounts.rs
programs/bc-solana-jump-router-sc/src/instructions/user/execute_swap_headless.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/constants.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/jump_pool_state.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/request_for_quote.rs
programs/bc-solana-jump-router-sc/src/utils/prop_amm/swap.rs
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [SVM Jump Router Market Account](#) code provided by [Securitize](#). The findings consist of 2 Informational.

6.1 Review Assumptions

The review assumed that the account layouts, ownership rules, account positions, and quote and swap behavior encoded by the changes are consistent with the relevant upstream AMM program and market-oracle program and their deployed versions. Those upstream programs and deployments were outside the scope of this engagement and were not independently reviewed or verified by Cyfrin.

6.2 Centralization Risks

The reviewed market-account support introduces no new privileged role or admin-controlled parameter. Existing administrators retain the ability to pause swaps, manage operators, and update pool, fee, and safety parameters. These controls predate the reviewed changes, and no new centralization risk attributable to this engagement's scope was identified.

6.3 Defensive Improvements

The review identified two Informational findings in the off-chain transaction tooling: the added market account can push some transactions beyond Solana's size limit, and pre-existing inconsistencies remain across the SDK and transport scripts. Both findings represent defensive improvements to the off-chain tooling.

Summary

Project Name	SVM Jump Router Market Account
Repository	bc-bd-router-sc
Commit	e87d62e327c9...
Fix Commit	d303871a59c2...
Audit Timeline	Sep 26th - Sep 27th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Informational	2	2	0
Gas Optimizations	0	0	0
Total	2	2	0

Summary of Findings

[I-1] The added market account pushes default swap transactions past the 1232-byte limit	Resolved
[I-2] Pre-existing off-chain SDK transaction-construction inconsistencies	Resolved

7 Findings

7.1 Informational

7.1.1 The added market account pushes default swap transactions past the 1232-byte limit

Description: The market-account change adds one unique read-only account to each market-backed swap. In a transaction that keeps its accounts in the static-key list, this adds 33 serialized bytes: 32 bytes for the public key and one byte for its index in the swap instruction.

```
// packages/jump-router-sdk/src/instructions/execute-swap.ts
.accountsStrict({
  operator: params.operator,
  swapAccounts: {
    // ...
    pool,
    market,
    instructions: SYSVAR_INSTRUCTIONS_PUBKEY,
    // ...
  },
})
```

The default transport flow keeps every account in that static-key list. The SDK exposes lookup tables through the optional `lutAddresses` parameter, and the CLI exposes them through the optional `--lookup-table-addresses` flag. The direct transport compiles a v0 message with an empty lookup-table list when the caller leaves this parameter unset. Fireblocks selects its legacy transaction path in the same situation. Both transports therefore rely on the caller to recognize the size requirement and provide a suitable lookup table.

```
// packages/jump-router-sdk/src/types/common-types.ts
lutAddresses?: PublicKey[];

// packages/jump-router-cli/src/direct-transport.ts
const lookupTableAccounts: AddressLookupTableAccount[] = [];
// The caller's tables are loaded here when lookupTableAddresses is populated.

const messageV0 = new TransactionMessage({
  payerKey,
  recentBlockhash: latestBlockhash.blockhash,
  instructions: ixes,
}).compileToV0Message(lookupTableAccounts);

// packages/jump-router-cli/src/fireblocks-transport.ts
if (!lookupTableAddresses?.length) {
  return sendAndConfirmTransaction(
    this.connection,
    new Transaction().add(...ixes),
    signers || [],
  );
}
```

These two behaviors combine at the transaction-size boundary. The supported transaction described below measured 1,222 bytes at the base revision. The new market account raises it to 1,255 bytes, past Solana's 1,232-byte limit for legacy and v0 transactions. RPC ingress rejects the transaction before the router instruction executes.

Solana's [format comparison](#) lists a 1,232-byte maximum for legacy and v0 transactions and a 4,096-byte maximum for v1. The direct transport builds v0 messages, while the default Fireblocks path builds legacy transactions. The lockfile resolves web3.js 1.98.4, and Solana's [v1 preparation guidance](#) confirms that the web3.js v1 series cannot build or send v1 transactions. The v1 limit therefore leaves both affected paths unchanged.

The CLI enables priority fees for this flow, so the 1,255-byte example contains both compute-budget instructions. Removing the 12-byte compute-unit-price instruction produces a 1,243-byte transaction, which also exceeds the limit.

A lookup table restores enough capacity when it contains at least two eligible accounts from this example. Moving only the new market account into a table increases the transaction to 1,258 bytes because the table metadata costs 35 bytes while replacing one 32-byte static key. Moving the market and one additional eligible account through the same table reduces the transaction to 1,227 bytes.

The CLI simulation path has a related consistency issue. It always calls `compileToV0Message()` with an empty lookup-table list, including when the operator supplies `--lookup-table-addresses`. The base revision already contained this behavior. The market-account change makes it relevant to this finding because a submission built with a suitable table fits at 1,227 bytes, while simulation rebuilds it in the 1,255-byte form.

```
// packages/jump-router-cli/src/commands/operator/execute-swap.ts
const executeSwapParams = {
  // ...
  lutAddresses: lookupTableAddresses,
  options: {
    withPriorityFee: true,
  },
};

if (simulateOnly) {
  // ...
  const messageV0 = new TransactionMessage({
    payerKey: user,
    recentBlockhash: latestBlockhash.blockhash,
    instructions: ixes,
  }).compileToV0Message();
}
```

The affected configuration uses:

- the user as fee payer;
- the same token program for both mints;
- an external fee manager and a distinct external fee collector;
- an existing external fee collector ATA;
- a 64-byte `external_id`; and
- the CLI's default priority-fee setting.

The resulting v0 transaction has two signatures, 27 static keys, two compute-budget instructions, and one `execute_swap` instruction with 28 account indices and 171 bytes of data:

signatures	$1 + 2 * 64 = 129$
v0 prefix and message header	4
static key vector	$1 + 27 * 32 = 865$
recent blockhash	32
instruction vector prefix	1
compute-unit-limit instruction	8
compute-unit-price instruction	12
execute_swap instruction	203
empty address-lookup vector	1

total	1,255 bytes

Agave enforces this boundary during transaction admission. The pinned Agave revision resolves [solana-packet 4.4.0](#), where `PACKET_DATA_SIZE` is asserted as 1,232 and defined as $1280 - 40 - 8$. For a legacy or v0 base64 transaction, `decode_and_deserialize()` selects that constant as `max_raw_size` and rejects a larger decoded payload. Its unit tests construct `PACKET_DATA_SIZE + 1` payloads and assert `InvalidParams`. The validator sanitization path selects the same constant for legacy and v0 before execution.

Impact: The default transport flow delivers a 1,255-byte transaction to RPC admission for the configuration above.

RPC admission rejects it before router execution, so balances remain unchanged. Operators can restore execution by supplying a lookup table containing enough eligible accounts, making this an availability and integration issue.

The simulation path produces the oversized static-key form of a transaction that the submission path can send with a suitable lookup table. Operators therefore receive a size rejection from `--simulate-only` instead of an execution result for the exact submission message.

Proof of Concept: The following construction runs entirely locally and uses the repository's IDL coder with synthetic public keys. Its only operation is transaction serialization.

The current IDL places 28 accounts in `execute_swap`. In the demonstrated configuration, payer equals user, `asset_program` equals `liquidity_program`, and the program account equals the invoked router program ID. These aliases leave 26 unique keys across the swap account list and invoked program. The Compute Budget program raises the compiled static-key count to 27. The construction below preserves those counts, signer roles, and the market's read-only role.

```
const assert = require('node:assert/strict');
const idl = require('./packages/jump-router-sdk/src/idl/solana-jump-router.json');
const { BN, BorshInstructionCoder } = require('@anchor-lang/core');
const {
  AddressLookupTableAccount,
  ComputeBudgetProgram,
  PublicKey,
  TransactionInstruction,
  TransactionMessage,
  VersionedTransaction,
} = require('@solana/web3.js');

const key = n => new PublicKey(Uint8Array.from({ length: 32 }, () => n));
const accounts = Array.from({ length: 25 }, (_, i) => key(i + 1));
const executeProgram = key(250);

const data = new BorshInstructionCoder(idl).encode('execute_swap', {
  swap_direction: { AssetForLiquidity: {} },
  amount_in: new BN(1),
  min_amount_out: new BN(1),
  nbbo_price: { 0: [new BN(1), new BN(0), new BN(0), new BN(0)] },
  expires_at: new BN(1),
  external_id: 'x'.repeat(64),
  external_fee_manager: {
    MbpsFeeManager: { 0: { numerator: 1, collector_wallet: key(200) } },
  },
});

const uniqueMetas = accounts.map((pubkey, i) => ({
  pubkey,
  isSigner: i < 2,
  isWritable: i !== 1 && i !== 24,
})).concat({ pubkey: executeProgram, isSigner: false, isWritable: false });

// payer/user and asset_program/liquidity_program account for the two repeated
// positions in the actual 28-account list. executeProgram is also present as
// the `program` account meta, so invoking it adds no further static key.
const currentMetas = uniqueMetas.concat(
  [2, 3].map(i => ({ pubkey: accounts[i], isSigner: false, isWritable: true })),
);

function compile(metas, { withPriorityFee = true, lookupTables = [] } = {}) {
  const swap = new TransactionInstruction({
    programId: executeProgram,
    keys: metas,
    data,
  });
}
```



```

const instructions = [ComputeBudgetProgram.setComputeUnitLimit({ units: 500_000 })];
if (withPriorityFee) {
  instructions.push(ComputeBudgetProgram.setComputeUnitPrice({ microLamports: 1 }));
}
instructions.push(swap);

const message = new TransactionMessage({
  payerKey: accounts[0],
  recentBlockhash: key(251).toBase58(),
  instructions,
}).compileToV0Message(lookupTables);

return {
  instructionDataBytes: data.length,
  staticKeys: message.staticAccountKeys.length,
  accountIndexes: message.compiledInstructions.at(-1).accountKeyIndexes.length,
  wireBytes: new VersionedTransaction(message).serialize().length,
};
}

const beforeMetas = currentMetas.filter(meta => !meta.pubkey.equals(accounts[24]));
const makeLut = addresses =>
  new AddressLookupTableAccount({
    key: key(249),
    state: {
      deactivationSlot: 18_446_744_073_709_551_615n,
      lastExtendedSlot: 0,
      lastExtendedSlotStartIndex: 0,
      authority: undefined,
      addresses,
    },
  });

const results = {
  beforeMarket: compile(beforeMetas),
  withMarket: compile(currentMetas),
  withoutPriorityFee: compile(currentMetas, { withPriorityFee: false }),
  marketOnlyLut: compile(currentMetas, { lookupTables: [makeLut([accounts[24]])] }),
  twoAccountLut: compile(currentMetas, {
    lookupTables: [makeLut([accounts[23], accounts[24]])],
  }),
};

console.log(results);
assert.equal(results.beforeMarket.wireBytes, 1222);
assert.equal(results.withMarket.wireBytes, 1255);
assert.equal(results.withoutPriorityFee.wireBytes, 1243);
assert.equal(results.marketOnlyLut.wireBytes, 1258);
assert.equal(results.twoAccountLut.wireBytes, 1227);

```

Output:

```

beforeMarket:      { instructionDataBytes: 171, staticKeys: 26, accountIndexes: 27, wireBytes: 1222 }
withMarket:        { instructionDataBytes: 171, staticKeys: 27, accountIndexes: 28, wireBytes: 1255 }
withoutPriorityFee: { instructionDataBytes: 171, staticKeys: 27, accountIndexes: 28, wireBytes: 1243 }
marketOnlyLut:     { instructionDataBytes: 171, staticKeys: 26, accountIndexes: 28, wireBytes: 1258 }
twoAccountLut:     { instructionDataBytes: 171, staticKeys: 25, accountIndexes: 28, wireBytes: 1227 }

```

Recommended Mitigation: Use one message-construction helper for simulation and submission. The helper should load the caller's lookup tables, compile the final message, and check its serialized size before submission. For an oversized message, return an actionable error that identifies the required LUT coverage.

Provide a canonical production lookup table, or select a suitable table automatically, for market-backed swaps that

approach the transaction limit. Include at least two eligible accounts from the demonstrated configuration because a market-only table adds three bytes to this transaction.

Securitize: Fixed in commit [d303871](#).

Cyfrin: Verified.

7.1.2 Pre-existing off-chain SDK transaction-construction inconsistencies

Description: The SDK and bundled direct transport contain three inconsistencies in fee-collector account handling and transaction fee-payer selection. These can block otherwise permitted collector configurations, reject concurrent first-use swaps, or charge network fees to a different signer than the SDK parameter describes.

All three behaviors are present at the feature base, 6fc34bb, and remain in the reviewed revision, e87d62e327c92b86b617eaa57f274f827429783c.

1. Admin SDK builders reject PDA fee collectors accepted by the program

The initialization, fee-manager update and pool-config update builders derive the protocol collector's associated token account (ATA) with `allowOwnerOffCurve = false`:

```
// packages/jump-router-sdk/src/instructions/update-fee-manager.ts:29
const feeCollectorAta = getAssociatedTokenAddressSync(
  jumpRouter.jumpRouterState.jumpPoolConfig.liquidityMint,
  feeCollectorWallet,
  false,
  liquidityProgram,
);
```

In the resolved SPL Token package, this throws `TokenOwnerOffCurveError` for a PDA before the instruction is produced. The [SPL Token API](#) documents that this flag controls whether PDA owners are allowed.

The on-chain collector constraints accept a configured nondefault public key without requiring a collector signature or an on-curve address. Both SDK swap builders already use `true` when deriving the same collector ATA.

Consequently, a valid treasury PDA distinct from the router custody authority cannot be configured through those admin builders. If configured through a lower-level instruction, the SDK pool-config update also fails during collector ATA derivation.

Relevant code:

- `packages/jump-router-sdk/src/instructions/initialize.ts:38`.
- `packages/jump-router-sdk/src/instructions/update-fee-manager.ts:29`.
- `packages/jump-router-sdk/src/instructions/update-jump-pool-config.ts:46`.
- `programs/bc-solana-jump-router-sc/src/instructions/admin/update_fee_manager.rs:25`.
- `packages/jump-router-sdk/src/instructions/execute-swap.ts:84` and `execute-swap-headless.ts:84`.

2. External collector ATA creation is not idempotent

Both swap builders query whether the external collector ATA exists. If it is absent, they prepend ordinary ATA creation:

```
// packages/jump-router-sdk/src/instructions/execute-swap.ts:105
if (!(await jumpRouter.provider.connection.getAccountInfo(externalFeeCollectorAta))) {
  preIxs.push(
    createAssociatedTokenAccountInstruction(
      params.payer ?? params.user,
      externalFeeCollectorAta,
      externalFeeCollectorWallet,
      liquidityMint,
      liquidityProgram,
    )
  );
}
```

```
    },  
  );  
}
```

The existence check occurs before transaction execution. Two swaps prepared while the same collector ATA is absent can therefore both include `Create`. Once the first transaction creates the account, the second transaction's creation instruction fails before the router swap instruction runs. A separate ATA creation between preparation and execution has the same effect.

Ordinary creation rejects an existing token account; idempotent creation accepts an existing ATA with the expected owner and mint. This distinction is visible in the [associated-token program processor](#).

Relevant code:

- `packages/jump-router-sdk/src/instructions/execute-swap.ts:105`.
- `packages/jump-router-sdk/src/instructions/execute-swap-headless.ts:105`.

3. The SDK's separate payer does not select the direct transport's network fee payer

Both swap parameter types describe payer as the payer for transaction fees. However, `executeSwap()` and `executeSwapHeadless()` append `payerKp` after `userKp`, while `DirectTransportV0` chooses the first signer as the message fee payer:

```
// packages/jump-router-cli/src/direct-transport.ts:34  
const payerKey = signers[0].publicKey;
```

When a caller supplies a distinct sponsor through `payer` and `payerKp`, also supplies `userKp`, and uses no custom signer override, the user remains the network fee payer. The selected sponsor still pays applicable ATA rent through the instruction's `payer` account.

This mismatch applies to the bundled direct transport. Custom transports may choose their message payer differently.

Relevant code:

- `packages/jump-router-sdk/src/types/common-types.ts:185` and `:226`.
- `packages/jump-router-sdk/src/jump-router.ts:303` and `:331`.
- `packages/jump-router-cli/src/direct-transport.ts:34`.
- `programs/bc-solana-jump-router-sc/src/instructions/swap_accounts.rs:44`.

Impact: These are off-chain compatibility, availability and API consistency issues:

- PDA fee collectors supported by the program cannot be configured through the affected SDK admin builders.
- Concurrent first-use swaps sharing an external collector can fail unnecessarily at ATA creation. Rebuilding after the account exists restores execution; the failed swap does not settle.
- A caller using the documented separate payer still charges the user's SOL balance through the direct transport. A user without sufficient SOL can fail despite a funded sponsor.

Recommended Mitigation:

1. Set `allowOwnerOffCurve = true` for protocol collector ATA derivation in all three admin builders, matching the program's recipient rules and the swap builders.
2. Use [createAssociatedTokenAccountIdempotentInstruction](#) when preparing an external collector ATA. This tolerates another valid creation after the RPC existence check.
3. Pass the intended network fee payer explicitly through the transport interface, or consistently place it first for transports using signer order. If `payer` is intended to cover ATA rent only, correct the parameter documentation and describe how callers select the network fee payer.

Securitize: Fixed in commit [8ce56a3](#).

Cyfrin: Verified.